Blink "Domain Driven Design"

Bridging the Gap Between Business and Code

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Blink "Domain Driven Design

Alberto Souza

2026-02-23

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Contents

O que este blink vai te ensinar? i

Sem profundidade no domínio, seu software será raso . 1

A distância entre quem sabe do negócio e quem escreve o código é o maior risco do seu projeto 3

A linguagem ubíqua é fácil de explicar, mas difícil de manter viva 5

Escrever código em português ou inglês importa menos do que você pensa 7

Contextos delimitados nascem de problemas, não de soluções 9

Delimitar contextos exige profundidade no problema e humildade para mudar de opinião 11

Nem tudo precisa ser construído do zero – e saber isso é estratégico 13

Só porque dois conceitos parecem iguais não significa que devem ser modelados da mesma forma 15

Entidades, objetos de valor e agregados emergem do código – não de diagramas 17

O DDD da massa: abraçar a tecnologia sem perder a alma do domínio 19

Resumo Final 21

Sobre este resumo 22

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

O que este blink vai te ensinar?

Imagine que você chega numa empresa nova, olha para o código de um sistema complexo e não consegue entender qual problema aquele software resolve. Os nomes das classes não fazem sentido, as abstrações parecem desconectadas do negócio, e cada pessoa da equipe usa palavras diferentes para descrever o mesmo conceito. Esse cenário é mais comum do que deveria – e é exatamente o tipo de problema que Domain Driven Design se propõe a resolver.

O treinamento "Domain Driven Design: As Partes que Importam", criado por Alberto Souza na Jornada Dev Eficiente, oferece mais de seis horas de teoria focada tanto na parte estratégica quanto na parte tática do DDD. Alberto traz uma abordagem honesta e prática, baseada em experiências reais de construção de produtos de educação e tecnologia, tangibilizando conceitos que no livro original de Eric Evans são abstratos o suficiente para gerar múltiplas interpretações.

A mensagem central é clara: o mais importante do DDD não são os patterns técnicos, mas sim a capacidade de se aprofundar no problema que você quer resolver e garantir que o software reflita esse entendimento de forma coesa.

Neste blink, você vai aprender:

- Por que se tornar especialista no domínio é mais importante do que dominar qualquer framework.
- Como construir e manter uma linguagem ubíqua que conecte todas as pessoas do time.
- De que forma identificar e delimitar contextos a partir dos problemas, e não das soluções.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

- Por que conceitos similares em contextos diferentes não devem ser modelados da mesma forma.
- Como entidades, objetos de valor, repositórios e agregados emergem do código na prática.
- O que significa abraçar a tecnologia sem perder a coesão com o domínio – o "DDD da massa".
- Por que a documentação e a manutenção contínua são essenciais para a saúde do software a longo prazo.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Sem profundidade no domínio, seu software será raso

O ponto de partida do Domain Driven Design não é nenhum pattern técnico. É um exercício de aprendizagem. Antes de escrever qualquer linha de código, você precisa entender profundamente o problema que o software se propõe a resolver. Alberto usa a taxonomia de Bloom para ilustrar isso: você precisa chegar pelo menos ao nível de "entender" – ser capaz de articular, associar, classificar e explicar o domínio com suas próprias palavras.

Quando Alberto quis criar exercícios conectados ao domínio de uma fintech chamada Monk, ele não simplesmente perguntou o que a empresa fazia. Ele investigou: por que adiantar pagamentos existe como serviço? Quais são as características de um adiantamento? Quem são as partes envolvidas? O que cada uma ganha e perde? Ele leu sobre o assunto antes de conversar com os especialistas. E depois da conversa, gravou vídeos explicando o que entendeu e enviou para validação.

Esse processo – estudar, tentar explicar, avaliar com especialistas, iterar – é o coração do aprofundamento de domínio. Não é algo que você faz uma vez e esquece. É um ciclo contínuo que vai refinando seu entendimento ao longo do tempo.

> *"Se você não parou para estudar sobre a parada, tem que parar. Tem uma pergunta que fica aqui para você: se você foi trabalhar na fintech, você fez curso relacionado às coisas que aquela fintech trabalha?"*

Quando chegar a um novo contexto de trabalho, busque as fontes teóricas sobre o domínio, estude, tente explicar com suas palavras, e avalie seu entendimento com quem é especialista. Quanto mais você entender do problema, melhor será o software que você vai

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

construir.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

A distância entre quem sabe do negócio e quem escreve o código é o maior risco do seu projeto

Em muitas equipes, existe uma divisão clara: de um lado, pessoas que são especialistas de domínio – product managers, analistas de negócio, product owners. Do outro, pessoas que são especialistas em tecnologia – engenheiras e engenheiros de software. Quando essa distância é grande, o resultado é previsível: reuniões intermináveis, traduções mal feitas de requisitos para código, e um software que não reflete o negócio.

Alberto argumenta que essa separação quebra a coesão do trabalho. É como se você tivesse estado e comportamento separados em objetos diferentes – a informação está num lugar e quem age sobre ela está em outro. O tempo inteiro você precisa alinhar, perguntar, validar. Isso gera duplicação de esforço e aumenta a chance de erros.

A solução proposta é direta: se você é engenheira ou engenheiro de software, seu objetivo deveria ser se transformar no máximo que puder em especialista de domínio. Isso não significa que você vai substituir a pessoa de produto, porque ela tem competências que vão além do conhecimento do domínio. Mas significa que, para modelar software orientado ao domínio, você precisa manjar do domínio.

> *"Quando você que está escrevendo o código não manja do domínio o máximo que pode, a sua chance de modelar orientado ao domínio começa a ir por água abaixo."*

Não espere que alguém traduza o negócio para você. Leia livros sobre o domínio, faça cursos, converse com especialistas. Quanto mais você diminuir a distância entre conhecimento técnico e con-

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

hecimento de negócio, mais coeso será o seu software.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

A linguagem ubíqua é fácil de explicar, mas difícil de manter viva

A ideia de criar uma linguagem ubíqua parece trivial: construir um vocabulário comum entre todas as pessoas envolvidas na construção do software. Especialistas de domínio, engenheiros, designers, suporte – todo mundo usando as mesmas palavras com as mesmas definições. Parece óbvio. A dificuldade está na execução contínua.

Alberto ilustra com o produto de educação que ele construiu. Naquele contexto, a palavra "task class" era fundamental – um conceito vindo do design instrucional 4C/ID que representa um conjunto de atividades relacionadas a um tipo de situação real. "Learning task" era outra expressão crucial. E termos como "autorregulação" e "objetivo de aprendizagem" precisavam estar no vocabulário de todos, não só dos especialistas em educação.

Um ponto importante é que a linguagem ubíqua não é composta apenas de palavras do negócio. Termos técnicos que são usados o tempo inteiro na comunicação da equipe também devem estar lá. Se o time fala constantemente sobre "acoplamento", "coesão" ou o nome do framework principal, essas palavras fazem parte do vocabulário compartilhado.

O grande desafio é manter isso vivo. Criar a primeira versão é fácil. Manter o documento atualizado conforme o software evolui, novos conceitos emergem do código, e pessoas entram e saem da equipe – isso exige disciplina e rituais.

> *"Criar a primeira versão disso, zero difícil. A dificuldade é você manter isso contínuo. Dificuldade grande."*

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Crie eventos regulares de manutenção da linguagem ubíqua – a cada duas semanas ou uma vez por mês. Cada contexto delimitado deve ter seu próprio vocabulário. Se uma palavra nova emerge do código, ela precisa alimentar a linguagem ubíqua, e vice-versa.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Escrever código em português ou inglês importa menos do que você pensa

Uma das discussões mais recorrentes em equipes brasileiras é se o código deve ser escrito em português ou inglês. Alberto traz uma perspectiva que corta esse debate pela raiz: o que importa é que o código reflita o jeito como a equipe realmente conversa sobre o domínio.

No produto de educação que Alberto construiu, a equipe percebeu que naturalmente misturava português e inglês nas conversas. "Task class" era sempre em inglês, porque vinha do design instrucional 4C/ID. Já "treinamento" era sempre em português. Forçar uma tradução – transformar "task class" em "classe de tarefas" ou "treinamento" em "training" – seria lutar contra a força natural da comunicação.

O problema real aparece quando não há combinado. Se a equipe decide escrever tudo em inglês, mas as palavras da linguagem ubíqua estão em português, alguém vai ter que traduzir. E se cada pessoa traduz de um jeito diferente, você perde a integridade com o modelo. "Curso" vira "course" para um e "training" para outro. Isso é uma janela quebrada que vai degradando o software aos poucos.

A solução é simples: declare as traduções na linguagem ubíqua. Se o time decidiu que o código será em inglês, as traduções oficiais de cada termo precisam estar registradas. Se decidiu misturar, registre o combinado. O pior cenário é deixar cada pessoa decidir sozinha.

> *"Não é que eu quero escrever tudo em português nem tudo em inglês. Eu quero escrever o código de um jeito que maximiza a*

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

> *coesão com o jeito que a gente conversa no dia a dia sobre esse problema."*

Registre na linguagem ubíqua como cada termo do domínio aparece no código. Se a equipe mistura idiomas naturalmente, deixe isso refletido no software. O importante é o combinado, não o idioma.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Contextos delimitados nascem de problemas, não de soluções

A parte de design estratégico do DDD é, na visão de Alberto, a mais importante e também a mais complexa. O conceito central é o bounded context – o contexto delimitado. E o erro mais comum é confundir contextos com soluções técnicas.

Alberto explica com um exemplo real: o grande problema que ele queria resolver era que equipes de engenharia produzem código cujo design deixa a desejar. Esse é o problema. A partir dele, surgiram dois subproblemas que delimitam contextos: as equipes não conhecem seu próprio nível de capacidade técnica (problema de avaliação), e as equipes não conseguem estudar com eficiência (problema de estudo eficiente). Cada um desses problemas é grande o suficiente para ter trabalho dedicado, com equipes, linguagem ubíqua e propostas de solução próprias.

O ponto crucial é que o contexto é delimitado pelo problema, não pela solução. Para o mesmo problema de avaliação, você pode ter múltiplas propostas de solução: autoavaliação, avaliações teóricas, avaliações práticas. Todas pertencem ao mesmo contexto. E dentro de cada proposta de solução, você pode ter um ou vários sistemas – um monólito, microsserviços, pacotes dentro de um projeto. A estrutura técnica é uma decisão posterior.

Alberto fez pouca distinção entre domínio, subdomínio e contexto delimitado de propósito. Para ele, a expressão "um contexto delimitado por um problema" captura a essência sem gerar confusão desnecessária entre termos que a própria literatura não consegue alinhar de forma consensual.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

> *"Quando você fala de contextos delimitados, pra mim esse é o melhor jeito de explicar: você tem um determinado problema, e em função dele, tem subproblemas que são importantes o suficiente para que sejam atacados de maneira supercoesa."*

Antes de pensar em sistemas, microsserviços ou arquitetura, defina com clareza o problema principal e os subproblemas que o compõe. Os contextos delimitados emergem dos problemas, e as soluções técnicas vêm depois.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Delimitar contextos exige profundidade no problema e humildade para mudar de opinião

Como você decide se algo é um contexto delimitado separado ou apenas uma parte de um contexto maior? Alberto admite que não existe resposta certa, mas oferece heurísticas práticas que ajudam a tomar essa decisão.

A primeira heurística é: se existem múltiplas maneiras complementares de atacar aquele subproblema, ele provavelmente merece ser um contexto próprio. No exemplo de avaliação, você pode ter autoavaliação, avaliações teóricas, avaliações práticas, avaliação de pares – todas são abordagens diferentes para o mesmo problema de "saber como você está indo". Essa riqueza de abordagens indica complexidade suficiente para um contexto dedicado.

Além disso, Alberto sugere definir limites explícitos para ajudar na decisão: número máximo de pessoas por contexto, número de propostas de solução, número de relações com outros contextos, complexidade da linguagem ubíqua. Quando várias dessas variáveis passam dos limites, é um sinal de que o contexto está grande demais e talvez precise ser dividido.

Mas há um aviso importante: começar com poucos contextos é mais seguro do que começar com muitos. Contextos demais geram mais necessidade de alinhamento, mais risco de vazamento de informação entre eles, e mais dificuldade de manter coesão. A sugestão de Alberto é começar conservador – aceitar contextos um pouco maiores no início – e dividir depois, conforme o entendimento se aprofunda.

Esse não é um exercício que você faz uma vez. Conforme o produto evolui e o entendimento do problema se aprofunda, novos contextos podem emergir e os existentes podem se reorganizar.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Isso é esperado.

> *"Não tem um jeito certo de fazer. Você precisa ter profundidade no entendimento do problema que quer resolver. Se você for raso, a sua solução vai acompanhar o seu nível de profundidade."*

Comece com menos contextos delimitados e aceite que eles sejam um pouco mais complexos. Divida depois, quando o entendimento do problema estiver mais maduro. Use limites explícitos – número de pessoas, propostas de solução, relações entre contextos – como sinais de alerta.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Nem tudo precisa ser construído do zero – e saber isso é estratégico

Quando você identifica propostas de solução dentro de um contexto delimitado, uma das decisões mais importantes é: construir ou comprar? Alberto considera essa escolha tão relevante quanto a definição dos próprios contextos.

No exemplo do produto de educação, a equipe precisava de um sistema de treinamento guiado baseado no design instrucional 4C/ID. Pesquisaram plataformas existentes – Open edX, entre outras – e concluíram que nenhuma atendia completamente ao que precisavam. Decidiram construir do zero. Essa decisão trouxe consigo responsabilidades: equipe para manter, bugs para corrigir, UX para amadurecer, evolução limitada ao conhecimento interno.

Já para avaliações teóricas, a situação era diferente. O Typeform resolvia o problema completamente. Construir um sistema de formulários melhor que o Typeform com uma equipe pequena seria irrealista. Então contrataram a solução e a integraram ao sistema de avaliações.

Alberto destaca que essa reflexão precisa considerar vários fatores: preço, nível de serviço, confiabilidade do fornecedor, durabilidade da solução. Escolher um serviço desconhecido e instável pode gerar mais problemas do que resolver. Por outro lado, não faz sentido reinventar a roda quando soluções consolidadas existem.

Um aspecto sutil é que, mesmo para problemas core, você pode usar soluções prontas. Avaliar pessoas é core no contexto de educação, mas o formulário em si é uma commodity. O que é core é o design das avaliações, a interpretação dos resultados, não a

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

infraestrutura de formulários.

> *"Eu gosto muito de pensar que eu só quero resolver alguma coisa que ninguém resolveu melhor do que a proposta que eu estou fazendo."*

Para cada proposta de solução dentro de um contexto, avalie se existe algo pronto que resolve bem o problema. Reserve a energia de construir do zero para aquilo que é verdadeiramente diferencial no seu produto. Soluções compradas podem servir até contextos core.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Só porque dois conceitos parecem iguais não significa que devem ser modelados da mesma forma

Uma das armadilhas mais comuns no desenvolvimento de software é a tentação de generalizar. Você vê um conceito parecido em dois sistemas diferentes e imediatamente pensa em criar uma abstração compartilhada. Alberto argumenta que, na maioria das vezes, isso é um erro.

No exemplo do contexto de estudo eficiente, a "task class" aparece tanto no sistema de treinamentos completos quanto no de práticas específicas. Em ambos, ela tem título, descrição, teorias, learning tasks. Mas há diferenças cruciais: no treinamento completo, uma task class pertence a um treinamento e só pode ser acessada em sequência. Na prática específica, a task class não pertence a nenhum treinamento e não tem restrição de acesso.

A modelagem ser parecida num momento do tempo não significa que ela vá continuar parecida. É apenas uma situação ocorrendo num determinado intervalo de existência entre dois sistemas. Tentar generalizar cria acoplamento: uma abstração compartilhada que precisa servir a dois contextos diferentes vai inevitavelmente acumular atributos opcionais, comportamentos condicionais, e complexidade desnecessária.

O mesmo vale para o conceito de "usuário", que aparece em praticamente todos os sistemas, mas com significados diferentes em cada um. No treinamento, é um aluno com seus treinamentos e respostas. Na avaliação, pode ser um funcionário que quer testar seu conhecimento. Modelar isso como uma única abstração compartilhada é pedir por problemas.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

> *"Não é porque os conceitos são similares que as modelagens vão ser sempre iguais. A modelagem ser parecida é só uma situação acontecendo num determinado intervalo de tempo."*

Resista à tentação de generalizar conceitos entre sistemas ou contextos diferentes. Prefira copiar código e adaptar a criar abstrações compartilhadas. O custo de manter algo genérico quase sempre supera o custo de manter coisas separadas.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Entidades, objetos de valor e agregados emergem do código – não de diagramas

A parte de design tático do DDD oferece building blocks para materializar em código os conceitos do domínio: entidades, objetos de valor, repositórios e agregados. Alberto enfatiza que esses conceitos são mais úteis quando emergem durante a implementação do que quando são definidos a priori.

Uma entidade é algo com um identificador único dentro da solução, que você vai querer persistir, recuperar e operar ao longo do tempo. No exemplo do treinamento, o título é o identificador natural – não podem existir dois treinamentos com o mesmo título. Alberto argumenta que o identificador pode ser composto por mais de um atributo: a task class, por exemplo, é identificada pela combinação de título e treinamento a que pertence.

Objetos de valor são definidos pela totalidade de suas características, sem identificador único. O "título ordenável" – que combina um título com uma posição – emergiu durante a implementação quando a equipe percebeu que os objetivos de aprendizagem precisavam de ordenação. Não existia esse conceito antes do código ser escrito.

Agregados aparecem quando você percebe que certas regras de negócio – invariantes – precisam ser garantidas através de um conjunto de entidades. Um treinamento não pode existir sem pelo menos uma task class. Cada task class precisa de pelo menos uma learning task obrigatória. Se você permite que uma learning task mude de tipo diretamente, pode quebrar essa regra. O agregado centraliza as operações para garantir a consistência.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

> *"Eu não acho que agregados são facilmente identificáveis. Vai ter situações em que eles vão emergir. O que eu acho mais importante é você manter a coesão entre o que está implementando e o problema que está resolvendo."*

Não gaste tempo excessivo tentando identificar todos os agregados antes de codar. Comece pelas entidades mais óbvias, implemente, e observe onde invariantes precisam ser protegidas. Os agregados vão emergir naturalmente das situações de inconsistência que você encontrar.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

O DDD da massa: abraçar a tecnologia sem perder a alma do domínio

Alberto cunhou a expressão "DDD da massa" para descrever uma abordagem que junta as ideias do Domain Driven Design com as facilidades que a tecnologia moderna oferece, sem lutar contra elas. A ideia é ser fortemente inspirado pelos conceitos do DDD, mas abraçar frameworks e ferramentas que aceleram o desenvolvimento.

Na prática, isso significa aceitar um pequeno acoplamento com a tecnologia em troca de produtividade. Por exemplo: usar anotações do Hibernate diretamente na classe que representa uma entidade do domínio, em vez de criar classes separadas para persistência e domínio. Adicionar um atributo "id" numérico gerado automaticamente, que não tem nada a ver com o domínio, mas facilita o trabalho com o banco de dados relacional.

Alberto reconhece que isso vai contra o idealismo de manter o domínio completamente desacoplado da infraestrutura. Mas argumenta que, na maioria dos cenários reais, o custo de manter essa separação total é maior do que o benefício. O importante é que o código continue revelando a intenção do domínio: os nomes das entidades, os métodos, as operações – tudo isso deve falar sobre o problema sendo resolvido.

A anatomia do código que Alberto defende tem mais palavras do domínio do que palavras de infraestrutura. Repositórios existem como ponto de partida para recuperar entidades, mas as operações são feitas sobre os objetos do domínio, não diretamente sobre a estrutura de dados. A diferença entre operar sobre entidades e operar diretamente com queries SQL é a diferença entre um design dirigido pelo domínio e um design dirigido pela in-

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

fraestrutura.

> *"Eu sou fortemente inspirado pelos conceitos do DDD porque acho que são importantes. Mas ao mesmo tempo eu não quero deixar de lado as tecnologias que facilitam minha vida e me permitem andar mais rápido, preservando a coesão."*

Não lute contra as facilidades que os frameworks modernos oferecem. Aceite pequenos acoplamentos com a tecnologia desde que o código continue coeso com o domínio. O importante é que qualquer pessoa do time consiga bater o olho no código e entender qual problema ele resolve.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Resumo Final

Domain Driven Design, nas partes que realmente importam, é sobre uma coisa acima de tudo: coesão. Coesão entre o problema que você quer resolver e o software que você constrói. Coesão entre as palavras que sua equipe usa e os nomes que aparecem no código. Coesão entre o entendimento do negócio e as decisões técnicas.

O caminho começa pelo aprofundamento no domínio – não existe atalho para isso. Você precisa estudar, conversar com especialistas, explicar com suas palavras, se avaliar. Depois, você constrói uma linguagem ubíqua que funcione como ponte entre negócio e código. Identifica os problemas que delimitam contextos e resiste à tentação de pensar em soluções antes de entender os problemas. Na hora de implementar, usa os building blocks do DDD – entidades, objetos de valor, agregados – mas deixa que eles emerjam do código ao invés de forçá-los a priori.

E, talvez o mais importante: aceita que tudo isso vai degradar com o tempo. O software vai ficando mais complexo, as pessoas mudam, o entendimento se aprofunda. Manter a saúde do sistema exige documentação, rituais de alinhamento, e a humildade de reconhecer que suas decisões de ontem podem precisar ser revisadas.

Comece pelo problema, não pela solução. Aprenda o domínio antes de modelar o código. Mantenha o vocabulário vivo. E lembre-se: qualidade não é negociável – ela é um investimento contínuo que exige carinho todos os dias.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89

Sobre este resumo

Este resumo foi criado a partir do treinamento **"Domain Driven Design: As Partes que Importam"**, ministrado por **Alberto Souza** na plataforma **Jornada Dev Eficiente**.

O treinamento oferece mais de seis horas de conteúdo teórico, cobrindo tanto a parte estratégica do DDD (aprofundamento de domínio, linguagem ubíqua, contextos delimitados) quanto a parte tática (entidades, objetos de valor, repositórios, agregados, serviços). Alberto traz uma abordagem baseada em experiências reais de construção de produtos de tecnologia e educação, tangibilizando os conceitos abstratos do livro original de Eric Evans.

A principal fonte de inspiração do treinamento é o livro **"Domain-Driven Design: Tackling Complexity in the Heart of Software"** de **Eric Evans**, complementado pela visão prática e opinativa de Alberto sobre como aplicar esses princípios no dia a dia de equipes que constroem software.

Adriano Avelino - adriano-st777@hotmail.com - IP: 179.0.127.89